

TUPLE

- Immutable
- Ordered
- Indexing
- Slicing

Tuple Initialization

t1 = ()

t1 = tuple()

× 1) t1 = (1)

✓ 2) t1 = 1,

✓ 3) t1 = (1,)

× 4) t1 = 1

⇒ "2" is a tuple

⇒ If comma is there, then without () it is considered as Tuple.
 (1, 2, 3, 4) = t1

⇒ t1 = 1, 2, 3, 4
print(type(t1))
↳ Tuple

Tuple packing

Is the process of combining multiple values into a tuple is called tuple packing.

Ex ⇒ t1 = (1, 7, 4.5, 8)

Tuple unpacking

• a, b, c = (10, 12, 11)
print(a) ⇒ 10
print(b) ⇒ 12
print(c) ⇒ 11

• a, *b, c = (1, 7, 9, 11, 12)

print(a) ⇒ 1

print(b) ⇒ [7, 9, 11]

print(c) ⇒ 12

⇒ Tuple Initialization,
Tuple Packing, Unpack,
Accessing Element,
sorted(), reversed()

• a, *b = (10, 20, 30)
print(a) ⇒ 10
print(b) ⇒ [20, 30]

List

⇒ When we unpack the tuple and if it contains multiple values assigned in it, then it will return in the form of 'LIST' not tuple

⇒ Accessing Elements in Tuple

^{-4 -3 -2 -1}
t1 = (1, 2, 3, 4)
_{0 1 2 3}

print(t1[1]) → 2
print(t1[3]) → 4
print(t1[-2]) → 3
print(t1[1:3]) → (2, 3)

t1 = tuple("hello")
↳ ("h", "e", "l", "l", "o")

• l1 = [10, 20, 30, 40, 50]

t1 = (1, 2, l1)

print(t1)

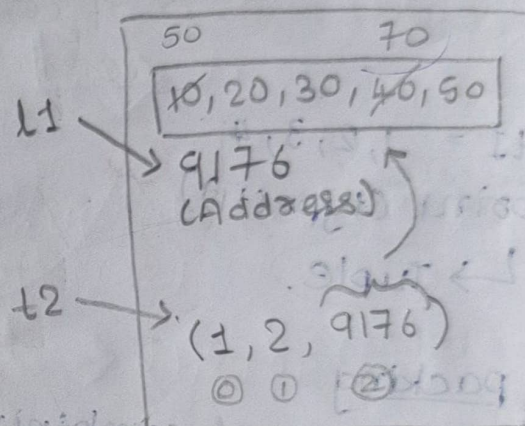
↳ (1, 2, [10, 20, 30, 40, 50])

l1[0] = 50

print(t1)

↳ (1, 2, [50, 20, 30, 40, 50])

(∴ We don't change tuple (Immutable),
we just modify the list (Mutable))



⇒ t1[2] = [80, 90] →

⇒ t1[2][3] = 70 →

print(t1)

↳ (1, 2, [50, 20, 30, 70, 50])

[you can modify the list,
but you can't replace list
⇒ tuple is immutable]

t1 = tuple(x*x for x in range(5))

print(t1) ⇒ (0, 1, 4, 9, 16)

∴ There is no tuple
comprehension
same like list
comprehension but,
you can write it
in concise way

Use with Tuple
sorted() sorted() reverse() reversed()

Tuple is immutable
⇒ Inplace method.